WAC
Windows Artefacts Collector

CENTRE D'EXCELLENCE CYBERDEFENSE AEROSPATIALE

Créé par le Centre d'Excellence Cyberdéfense Aérospatiale (CEC)

Face à KAPE et EZ
WAC (Windows Artefacts Collector)

Logiciel de prélèvement d'artefacts Windows pour l'investigation numérique



ARTEFACTS SYSTÈME

FICHIERS UTILISATEUR

PREUVES NUMÉRIQUES

UN OUTIL AU SERVICE DE L'INVESTIGATION NUMÉRIQUE

WAC — Windows Artefacts Collector

Face à KAPE et aux outils d'Eric Zimmerman

Version du logiciel	2.0.0
Édition du document	octobre 2026
Éditeur	Centre d'Excellence Cyberdéfense Aérospatiale (CEC)
	École de l'air et de l'espace, base aérienne 701, Salon-de-Provence

Ce document confronte WAC aux outils de référence de la communauté : KAPE pour la collecte, les outils en ligne de commande d'Eric Zimmerman pour le décodage. Sur les mêmes pièces d'une même machine, il compare les temps d'exécution, ce que chacun prélève, la justesse des valeurs — chaque écart étant tranché par une référence indépendante —, les formats de sortie, et les traces que chacun laisse sur la machine examinée. Il dit ce que la confrontation a appris à WAC, et ce qu'elle lui reproche encore.

Table des matières

1 La démarche **2**

1.1 Ce qui est comparé . . . 2

1.2 Qui a raison quand ils diffèrent . . . 2

1.3 Le banc d'essai . . . 3

2 Synthèse **4**

3 Temps d'exécution **6**

3.1 La méthode . . . 6

3.2 Collecte . . . 6

3.3 Conversion et décodage . . . 7

3.4 Collecte depuis un même état . . . 9

4 Ce que chacun prélève **10**

4.1 Les mêmes pièces, des fichiers différents . . . 10

4.2 Ce qui est consigné de chaque pièce . . . 10

5 Traces laissées sur la machine examinée **12**

5.1 La mesure . . . 12

5.2 Ce que chacun laisse . . . 12

5.3 Ce que la mesure a corrigé dans WAC . . . 14

6 Précision **15**

6.1 En faveur de WAC . . . 15

6.2 En faveur des outils EZ . . . 16

7 Formats de sortie **18**

7.1 Ce que produit chaque outil . . . 18

7.2 Ce qui les distingue . . . 18

8 Ce que la confrontation a appris à WAC **20**

9 Limites de la comparaison **22**

CHAPITRE 1

La démarche

1.1 Ce qui est comparé

Deux questions, deux comparaisons.

Collecte : WAC face à KAPE
Les deux outils collectent la même machine l'un après l'autre ; on compare ce que chacun copie, le temps qu'il y met, et l'empreinte SHA-1 des fichiers qu'ils copient tous deux.

Traces laissées : WAC, KAPE et les outils EZ
Chacun est lancé depuis le même état d'une même machine, sous Sysmon ; on relève ce qu'il y écrit, ce que Windows y écrit à cause de lui, et ce qu'un outil de surveillance en voit (chapitre 5).

Décodage : WAC face aux outils EZ
Sur les *mêmes pièces* — la consigne d'une collecte WAC —, WAC convertit la collecte entière et l'outil d'Eric Zimmerman correspondant décode, sur une copie, les pièces de son artefact ; les deux sorties sont comparées enregistrement par enregistrement et champ par champ.

1.2 Qui a raison quand ils diffèrent

Un écart entre deux outils ne dit pas lequel se trompe. Chaque écart a donc été tranché par une troisième source, indépendante des deux :

— Windows lui-même : le XML qu'il rend pour un événement (`Get-WinEvent`), la date d'un fichier, l'identifiant d'objet NTFS (`fsutil objectid`), le registre en ligne, l'empreinte d'un fichier (`Get-FileHash`) ;
— une autre implémentation : `dissect.esedb` (Fox-IT) pour les bases ESE ;
— le format lui-même : numéros de séquence et sommes Marvin32 des journaux de ruche, lus octet par octet ;
— le recoupement de deux artefacts de la même machine : le Prefetch pour l'exécution, l'horodatage d'une ligne pour ses autres dates.

Chaque écart est ensuite classé : *défaut de WAC* (corrigé), *erreur de l'outil EZ* (vérifiée), ou *convention* de l'un des deux — deux formes d'une même valeur.

1.3 Le banc d’essai

Machine	Windows 11 Professionnel (build 26200), machine virtuelle
Outils EZ	version 2026.5.0 (.NET 9.0.20)
KAPE	version 1.3.0.2, cible `KapeTriage`
WAC	version 2.0.0 et ses corrections issues de cette confrontation
Mesure des traces	clone de la machine, sans réseau, figé ; Sysmon 15.22 ; outils sur une clé USB (K:), une exécution par outil

Tout se rejoue : `vmtest/ez-compare.sh <artefact>` chronomètre l’outil EZ sur une copie des pièces et compare sa sortie à la conversion de WAC ; `vmtest/lib/timing-table.py` met les temps de chaque étape en regard ; `vmtest/kape-compare.sh` fait de même pour la collecte ; `vmtest/footprint/run-footprint.sh` rejoue la mesure des traces et `footprint-diff.py` l’analyse. Les outils EZ ont été lancés avec leur pleine précision de date (`--mp`, ou `--dt` au format à sept décimales quand `--mp` ne suffit pas), pour ne pas leur imputer une perte de précision qu’un réglage évite.

CHAPITRE 2

Synthèse

Pour chaque artefact, les sorties de WAC et de l'outil EZ sont appariées enregistrement par enregistrement, puis comparées champ par champ. La dernière colonne donne chaque écart trouvé et qui a raison, établi par une source indépendante (chapitre 6) ; un champ qui n'y figure pas a la même valeur des deux côtés.

Artefact — outil	Comparés	Champs comparés	Écarts, et qui a raison
$MFT — MFTECmd	361 920 entrées	chemin, état (supprimé, dossier), 8 dates, taille, entrée MFT	tailles de 1 491 fichiers compressés (WOF), 8 fichiers ouverts, 1 fichier supprimé : WAC, comme Windows
EVTX — EvtxECmd	211 525 événements	identifiant, date, fournisseur, machine, données de l'événement	10 valeurs : WAC, comme Windows ; 435 événements Perflib qu'EvtxECmd ne lit pas ; journaux vidés qu'il ignore
Prefetch — PECmd	478 fichiers	exécutable, empreinte, exécutions et leurs dates, volumes, fichiers chargés	empreinte sans son zéro de tête chez PECmd (23) ; répertoires de deux volumes collés (1)
Raccourcis — LECmd	26 fichiers	cible, dates au 100 ns, taille, volume, machine, arguments	aucun
Jump lists — JLECmd	60 entrées	chemin, compteur, épinglage, dates, raccourci de chaque entrée	aucun
Shellbags — SBECmd	46 dossiers	nom, rang, dates de la clé et du dossier, entrée MFT	NodeSlot faux au premier niveau chez SBECmd (9) ; « exploré » toujours faux
Amcache — AmcacheParser	1 479 entrées	les 124 colonnes communes des 7 tables	3 450 valeurs absentes de la ruche, inventées par AmcacheParser ; SHA-1 partiel non signalé

Artefact — outil	Comparés	Champs comparés	Écarts, et qui a raison
ShimCache — AppCompat-CacheParser	367 entrées	chemin, date, rang	aucun ; l'indicateur « exécuté » d'AppCompatCacheParser, trompeur, n'est pas relevé
Corbeille — RBCmd	66 fichiers	chemin d'origine, taille, date de suppression	aucun
SRUM — SrumECmd	91 008 lignes	les 5 tables, toutes leurs colonnes	16 lignes d'énergie fausses chez SrumECmd (dissect.esedb) ; base non fermée qu'il refuse
Registre — RECmd	9 artefacts	UserAssist, BAM, USBSTOR, comptes, Run, MuiCache, chemins tapés, recherches, documents de confiance	4 ruches ramenées à un état ancien par RECmd ; un compte supprimé encore listé
Collecte — KAPE	1 416 fichiers pris des deux côtés	empreinte SHA-1	23 différents : fichiers que Windows a réécrits entre les deux collectes

À retenir

Sur tout ce que les deux côtés lisent, WAC donne les mêmes valeurs que les outils de référence, ou des valeurs plus justes, établies à chaque fois par une source indépendante. La confrontation a aussi révélé 28 défauts de WAC, tous corrigés (chapitre 8), et des lacunes de couverture, toutes comblées depuis (section 6.2).

Lancé depuis une clé, WAC n'écrit rien sur la machine examinée : ni fichier, ni valeur de registre, ni événement — seules restent les traces que Windows laisse de tout programme lancé (Amcache, ShimCache). KAPE et les outils EZ, des programmes .NET, écrivent dans le registre de l'utilisateur et, pour les modules de KAPE, des fichiers sur le volume système (chapitre 5).

CHAPITRE 3

Temps d'exécution

3.1 La méthode

WAC ne se mesure pas artefact par artefact. Une conversion lancée pour un seul artefact lui fait payer, à chaque fois, ce qu'il ne paie qu'une fois en vrai : vérifier le sceau et les empreintes de la consigne, reconstruire la copie de travail, réparer les ruches. Une collecte lancée pour un seul artefact lui fait rouvrir et parcourir le volume à chaque fois. Les temps ci-dessous viennent donc d'*une* collecte complète et d'*une* conversion complète. WAC y note la durée de chacune de ses étapes, dans `investigation.json` et `conversion.json`. Chaque artefact est mis en face de l'outil EZ qui le décode. Les étapes que les autres outils n'ont pas sont données à part. Le rapprochement est fait par `vmtest/lib/timing-table.py`.

3.2 Collecte

KAPE (cible `KapeTriage`) et WAC collectent la même machine, l'un juste après l'autre.

Outil, réglage	Durée	Fichiers	Volume
KAPE, `KapeTriage`	44 s	1 427	0,96 Go
WAC, sans les exécutables (`binary: none`)	41 s	2 178	2,62 Go
WAC, kit livré (`binary: unverified`)	488 s	72 887 consignés	3,66 Go

Sans les exécutables, WAC est un peu plus rapide que KAPE et ramène 2,7 fois plus de données. Ses étapes, mesurées dans cette exécution :

Étape de WAC	Durée	Chez KAPE
Contrôles : version de Windows, droits, place libre sur le support	0,00 s	aucun contrôle de place
Observation de la machine en marche : sessions, processus, réseau, services, horloge	0,16 s	—
Ruches du système, en lecture brute NTFS ; le volume est ouvert une fois, et ses répertoires mis en cache pour toute la collecte	2,09 s	copie, lecture brute pour les fichiers verrouillés
Ruches des utilisateurs	0,23 s	idem

Étape de WAC	Durée	Chez KAPE
Fichiers des artefacts (journaux, Prefetch, raccourcis, navigateurs…)	18,10 s	idem
$MFT, $UsnJrnl, $LogFile, $Boot, $Secure:$SDS	6,03 s	idem
Recherche des volumes FAT et exFAT (clés USB) et lecture de leurs métadonnées	0,02 s	—
Fichiers de ressources des fournisseurs que citent les journaux, pour le texte des événements	11,15 s	—
Scellement : manifeste, empreinte SHA-256 du manifeste	0,09 s	journal de copie
Hors étapes : démarrage, écriture du journal d'investigation	3,1 s	

Les trois empreintes de chaque pièce (MD5, SHA-1, SHA-256) et ses dates NTFS sont prises pendant la lecture même : le scellement n'a plus qu'à écrire le manifeste. Les étapes propres à WAC — contrôles, observation de la machine, volumes FAT, ressources des fournisseurs, scellement — coûtent 11,4 s, dont 11,2 s pour les ressources des fournisseurs. Ce sont elles qui permettent, sur le poste d'analyse, de rendre le texte de chaque événement comme Windows l'affiche.

Avec les exécutables (réglage du kit livré), la collecte prend 488 s, dont 389 s pour lire chaque exécutable, bibliothèque, pilote et script du volume. WAC en vérifie la signature Authenticode, en mémoire, contre son jeu de confiance, et ne copie que ceux dont la signature ne tient pas. Les 54 467 fichiers lus se répartissent ainsi :

— 28 180 authentifiés comme Microsoft et 328 validés par une chaîne tierce : empreintes consignées, fichiers non copiés ;
— 12 691 fichiers différentiels de Windows (MSDELTA), qui ne sont pas des exécutables : empreintes consignées, non copiés ;
— 2 599 dont la signature ne tient pas, copiés (771 Mo) ;
— 10 669 au contenu identique à une pièce déjà copiée : consignés, renvoyés à cette pièce, non copiés une seconde fois.

Les 43 s de l'observation des processus viennent, dans ce réglage, des empreintes de chaque module chargé, et les 5 s du jeu de confiance de son chargement. KAPE ne fait aucun de ces contrôles : il copie les fichiers de ses cibles sans les examiner.

3.3 Conversion et décodage

La conversion de WAC lit la consigne d'une collecte, sur le poste d'analyse. Chaque outil EZ lit une copie des mêmes pièces : l'outil est chronométré seul, la copie non. Cette copie n'est pas une précaution de forme. Lancé sur la consigne elle-même, SrumECmd y a écrit (son `Jet Init` pose un point de contrôle et des journaux à côté de la base), et WAC a ensuite refusé de convertir une consigne qui ne correspondait plus à son manifeste.

Artefact	wac	wac --binary	Outil EZ	Outil
$MFT (C : et G:)	4,14 s	4,02 s	9,2 s	MFTECmd (C : seul)
Journaux d'événements	10,65 s	9,97 s	24,6 s	EvtxECmd
Prefetch (478)	0,33 s	11,66 s	2,4 s	PECmd
Raccourcis	0,03 s	0,04 s	1,6 s	LECmd
Jump lists	0,03 s	0,04 s	0,6 s	JLECmd
Shellbags	0,01 s	0,01 s	0,9 s	SBECmd
ShimCache	0,01 s	0,39 s	1,2 s	AppCompatCache-Parser
Amcache	0,07 s	8,96 s	0,7 s	AmcacheParser
Corbeille	0,07 s	0,01 s	0,5 s	RBCmd
SRUM	1,77 s	1,72 s	2,5 s	SrumECmd
Registre (lot Kroll)	0,26 s	1,43 s	10,2 s	RECmd
Somme	17,4 s	38,3 s	54,4 s	les 11 outils

À ces étapes s'ajoutent celles que wac paie une fois pour tous les artefacts, et que les outils EZ n'ont pas :

Étape commune de wac	Durée
Vérification du sceau et des empreintes de chaque pièce	10,9 s
Reconstruction de la copie de travail depuis la consigne (2,1 Go), chaque copie vérifiée par son empreinte	45,1 s
Réparation des ruches de la copie de travail (rejeu de leurs journaux)	0,5 s
Fuseau, page de codes, profils et noms des comptes du suspect	0,0 s
Total	56,5 s

La conversion entière dure 80 s : les 56 s communs, les 17,4 s des onze artefacts comparés, et 6,4 s pour les 38 autres, que les outils EZ ne lisent pas (journal des modifications, tâches planifiées, caches de miniatures, connexions RDP, courrier, navigateurs…). Les onze outils EZ prennent ensemble 54 s, sans copie de travail ni vérification de la consigne.

— Une fois la copie de travail faite, chaque décodage de wac est plus rapide que l'outil EZ correspondant : les ruches sont lues une fois, pour tous les artefacts du registre.

— L'étape la plus lourde est la copie de travail. Elle est la condition d'une analyse qui ne touche jamais la consigne, et la même copie est nécessaire aux outils EZ, comme SrumECmd l'a montré.

— Avec `--binary`, WAC calcule l'empreinte de chaque fichier que citent les artefacts : +11,3 s pour le Prefetch (les bibliothèques chargées), +8,9 s pour l'Amcache. Les outils EZ ne le font pas.

3.4 Collecte depuis un même état

Mesures de la comparaison des traces (chapitre 5) : chaque outil part du même état figé de la machine, lancé depuis la clé ; la durée est celle de la tâche qui le lance, entre parenthèses celle que l'outil annonce.

Outil, réglage	Durée	Pris	Sur la clé
KAPE, `!SANS_Triage`	127 s (115 s)	2 291 fichiers	2 286 fichiers, 1,25 Go
KAPE, `!SANS_Triage` et modules `!EZParser`	235 s (223 s)	2 288 fichiers, 14 outils	2 345 fichiers, 1,63 Go
Outils EZ sur la machine vivante (6 outils)	132 s	$MFT, journaux, ruches, Prefetch, Amcache, ShimCache	36 CSV, 0,55 Go
WAC, collecte (réglage par défaut)	592 s (588 s)	11 830 pièces, 67 882 consignées	8 240 fichiers, 4,9 Go

La collecte de WAC est la plus longue parce qu'elle lit *chaque* exécutable de la machine : la conversion se fera ailleurs, et elle ne sait pas encore lesquels les artefacts citeront. Elle en calcule les empreintes, vérifie leur signature et ne copie que ceux dont la signature ne tient pas ; d'où 67 882 pièces consignées pour 11 830 copiées. La conversion, sur le poste d'analyse, ne relit rien de la machine (chapitre 5). KAPE copie les fichiers de ses cibles sans les examiner.

CHAPITRE 4

Ce que chacun prélève

4.1 Les mêmes pièces, des fichiers différents

Dernière comparaison : KAPE avec sa cible `KapeTriage`, et WAC sans les exécutables (`binary: none`), l'un juste après l'autre ; les fichiers sont appariés par leur chemin d'origine (manifeste de WAC, journal de copie de KAPE). KAPE prend 1 421 fichiers, WAC 2 206 ; 1 416 sont pris par les deux.

KAPE ne prend que 5 fichiers que WAC ne prend pas : des `desktop.ini`, qui ne fixent que l'icône et le nom affiché d'un dossier. WAC prend en plus 790 fichiers : les fichiers de ressources des fournisseurs d'événements (286), qui donnent le texte des événements ; Defender (123) ; les raccourcis hors de Recent (76) ; le courrier de Thunderbird, les caches de miniatures de l'Explorateur et l'index de recherche de Windows (183 en tout) ; les rapports d'erreur (32) ; d'autres ruches et des métadonnées NTFS ; BITS, WMI, DPAPI, hosts, setupapi, les volumes FAT, l'état de la machine en marche.

Sur les 1 416 fichiers communs, 1 393 ont la même empreinte SHA-1. Les 23 autres sont des fichiers que Windows réécrit sans cesse — $MFT, journal des modifications, Prefetch, journaux de Defender, SRUM — et qu'il a modifiés entre les deux collectes.

4.2 Ce qui est consigné de chaque pièce

	WAC (manifeste)	**KAPE (journal de copie)**
Empreintes	MD5, SHA-1, SHA-256	SHA-1
Dates du fichier	les 4 de `$STANDARD_INFORMATION` et les 4 de `$FILE_NAME`, au 100 ns, en UTC et en heure locale du suspect	3 de `$STANDARD_INFORMATION` (création, modification, accès), au 100 ns, en UTC, sans le changement de l'enregistrement MFT
Emplacement	numéro d'enregistrement MFT	chemin seul
Méthode	dite pour chaque pièce (lecture brute NTFS, observation vivante)	copie ordinaire ; lecture brute « différée » pour les fichiers verrouillés (181 sur 2 292)
Échec	code et cause dans le manifeste, pièce par pièce	console et journal

	WAC (manifeste)	**KAPE (journal de copie)**
Intégrité	manifeste scellé (SHA-256), revérifié à chaque conversion	—

À retenir

Les dates de `$FILE_NAME` sont celles qu'un intrus ne réécrit pas avec les fonctions ordinaires : un fichier antidaté (« timestomping ») garde dans `$FILE_NAME` sa vraie date de création. Le journal de KAPE ne les a pas ; le manifeste de WAC les donne pour chaque pièce, avec l'enregistrement MFT qui permet de la retrouver dans `mft.json`.

CHAPITRE 5

Traces laissées sur la machine examinée

5.1 La mesure

Un clone de la machine de test, sans réseau, est figé avec Sysmon installé ; les outils sont sur une clé USB, lettre K:. Chaque exécution part de ce même état : le disque système et la clé sont des couches neuves sur les images figées. Le démarrage passé, chaque outil est lancé de la même façon — une tâche planifiée de l'utilisateur, avec ses droits élevés, comme un examinateur depuis une invite élevée.

Les fichiers
Le numéro USN du journal des modifications de C : est relevé juste avant et juste après l'outil : tout fichier créé, modifié, renommé ou supprimé entre les deux y figure, même créé puis supprimé pendant l'exécution. Les relevés sont écrits sur la clé, après la borne de fin.

L'auteur
Sysmon donne l'arbre des processus de l'outil et ce que chacun a écrit, fichiers et registre. Une écriture d'un autre processus qui nomme l'outil (Amcache, Prefetch, BAM) est une réaction de Windows à son lancement.

Le témoin
Une exécution qui ne lance rien (`ver`) mesure le lanceur et l'activité propre de Windows ; ce qu'elle montre aussi n'est pas imputé à l'outil.

Le différé
Le ShimCache n'est écrit qu'à l'arrêt : chaque exécution se termine par un arrêt propre, un redémarrage et sa lecture.

5.2 Ce que chacun laisse

Outil	Fichiers écrits sur C :	Registre écrit	Réactions de Windows
WAC (collecte)	aucun	aucun	Amcache : une entrée pendant l'exécution ; ShimCache : une entrée à l'arrêt

Outil	Fichiers écrits sur C :	Registre écrit	Réactions de Windows
KAPE, cibles	aucun	4 valeurs `ZoneMap` d'Internet Settings dans la ruche de l'utilisateur ; 169 ouvertures en création (magasins de certificats, stratégies)	Amcache et ShimCache pour `kape.exe`
KAPE, cibles et modules	28 : 14 dans `ProgramData\Microsoft\NetFramework\BreadcrumbStore`, 14 dans le Temp de l'utilisateur (`system-commandline-sentinel-files`)	les mêmes 4 valeurs, réécrites par 14 des 15 processus (136 écritures) ; environ 2 600 ouvertures en création	Amcache et ShimCache pour KAPE et 14 outils
Outils EZ, en direct	aucun	aucune valeur ; 54 ouvertures en création (stratégies, `Tcpip`, `Internet Settings`)	Amcache et ShimCache pour les 6 outils

Aucun processus d'aucun outil n'a fait écrire d'événement dans les journaux de Windows : ceux de la fenêtre de mesure sont les ouvertures de session des services, Defender, la stratégie de groupe, communs au témoin ou sans lien avec l'outil. Tous lisent les volumes en brut, ce que Sysmon relève (`RawAccessRead`) et qu'un EDR peut signaler.

Ouverture en création
`RegCreateKeyEx` crée la clé si elle manque et l'ouvre sinon ; Sysmon ne distingue pas les deux cas. Ce sont les bibliothèques .NET qui ouvrent ainsi les magasins de certificats et les clés de stratégie. Une clé créée change de date ; une clé seulement ouverte, non.

Les valeurs `ZoneMap`
Écrites par l'initialisation de WinINet dans chaque processus .NET qui la charge, elles modifient la ruche `NTUSER.DAT` de l'utilisateur examiné, donc son journal et la date de la clé.

Les fichiers .NET
Le runtime laisse une « miette » par assemblage chargé dans `BreadcrumbStore`, et la bibliothèque System.CommandLine un fichier par outil dans le Temp de l'utilisateur. Lancés seuls, les outils EZ n'en ont pas laissé ; lancés par KAPE, si.

Prefetch et BAM
Aucun des 22 programmes lancés depuis la clé USB n'a reçu de fichier Prefetch ni de valeur BAM, quand `cmd.exe`, lancé du disque système par la même tâche, a sa valeur BAM. Un outil copié sur la machine et lancé de là en recevrait.

Le prérequis .NET
KAPE et les outils EZ demandent le runtime .NET 9, présent sur le clone. Sur une machine qui ne l'a pas, l'installer écrit des centaines de fichiers et de clés ; WAC est un exécutable autonome.

5.3 Ce que la mesure a corrigé dans WAC

WAC effaçait l'écran au démarrage par `system("cls")`, qui lance `cmd.exe` : un processus créé sur la machine examinée et le Prefetch de `CMD.EXE` mis à jour, à chaque collecte. L'écran est désormais effacé par l'API de la console, WAC n'importe plus aucune fonction capable de lancer un processus, et le script de compilation refuse une version qui en importerait une.

> **À retenir**
> Chaque trace de WAC est expliquée : il ne lance aucun processus, n'écrit aucun fichier sur la machine, aucune valeur de registre, aucun événement. Restent celles que Windows laisse de tout programme lancé — une entrée Amcache, une entrée ShimCache à l'arrêt —, inévitables et décrites dans le dossier d'innocuité, et la lecture brute des volumes, visible d'un EDR.

CHAPITRE 6

Précision

6.1 En faveur de WAC

Journaux de ruche rejoués dans l'ordre
Les journaux `.LOG1` et `.LOG2` gardent, derrière les entrées récentes, les restes d'une génération précédente. RECmd les applique dans l'ordre du fichier, donc après les plus récentes. Il ramène ainsi SYSTEM, SECURITY, DEFAULT et une ruche de profil de service à un état plus ancien : dans SYSTEM, la séquence 0x3618 au lieu de 0x3620, et quatre dernières exécutions BAM affichées 13 minutes trop tôt. WAC trie les entrées par séquence et s'arrête à la rupture de chaîne. Les journaux eux-mêmes le vérifient, sommes Marvin32 comprises.

Événements des journaux vidés
Un journal vidé garde ses enregistrements : WAC les relit et les marque `EvtRecovered`, sans jamais les présenter comme vivants ; EvtxECmd ne les voit pas.

Valeurs d'événements comme Windows les rend
Hexadécimal `0x3e7`, GUID entre accolades, booléens `true`, binaire `69060000`, dates avec leur `Z`, marques U+200E conservées : les formes qu'affiche l'Observateur d'événements et que l'on recherche. EvtxECmd en change huit, et se trompe sur dix valeurs, par exemple un booléen `False` que Windows rend `true` ; chacune est tranchée par le XML que rend Windows (`Get-WinEvent`) sur le même journal.

Rien d'inventé
Une valeur absente de la source n'est pas émise. AmcacheParser écrit 3 450 valeurs par défaut absentes de la ruche (`IsPeFile: False` pour un exécutable PE), SrumECmd 10 936 (0 ou -1).

SHA-1 partiel de l'Amcache signalé
Pour un fichier de plus de 30 Mio, Amcache ne hache que les 31 457 280 premiers octets (mesuré sur `mrt.exe`). WAC le dit (`AmcacheSha1CoversBytes`) ; sans cela, la comparaison avec le SHA-1 du fichier entier ferait conclure à tort à une modification.

Pas de champ trompeur
L'indicateur « exécuté » du ShimCache, que donne AppCompatCacheParser, dit « non » pour 105 des 117 binaires dont le Prefetch prouve l'exécution. WAC ne le relève pas : ce « non » ferait conclure qu'un programme n'a jamais tourné.

Précision au 100 ns
WAC donne chaque date à la précision de sa source. Par défaut, PECmd, LECmd et SrumECmd arrondissent à la seconde, et LECmd ignore `--mp` dans son CSV.

Heure locale datée
Chaque date est donnée en UTC et en heure locale du suspect, avec le décalage en vigueur à cette date. Les outils EZ ne donnent que l'UTC, sans le signaler par un `Z`.

Lectures d'autrui vérifiées
Les « FirstInteracted » et « LastInteracted » de SBECmd, le « CreatedOn » des comptes chez RECmd sont des lectures de dates d'écriture de clés. WAC publie ces dates elles-mêmes : chacun en tire la lecture qu'il veut, sans en hériter une toute faite. SBECmd donne en outre un `NodeSlot` faux (0) aux dossiers de premier niveau.

Erreurs de lecture ESE de SrumECmd
16 lignes d'énergie sur 1 370 portent le contenu d'une autre ligne ou un identifiant absent de la table ; dissect.esedb donne raison à WAC.

Base ESE non fermée proprement
Une base SRUM prise sur une machine en marche n'est pas fermée proprement. SrumECmd s'arrête sur elle ; WAC rejoue ses journaux sur la copie de travail (`esent.dll`), puis la lit.

Comptes supprimés
Le banc supprime un compte (`wacrdp`) après s'en être servi. RECmd le liste encore, d'une SAM ramenée à un état antérieur ; WAC ne le donne pas, et `Get-LocalUser` non plus.

Collecte
Chaque pièce de WAC porte MD5, SHA-1 et SHA-256, les huit dates NTFS de son original et son entrée MFT, dans un manifeste scellé. KAPE donne un SHA-1 et trois dates.

6.2 En faveur des outils EZ

Couverture
Il n'y a plus de lacune ouverte. Les clés du lot Kroll que WAC ignorait sont désormais lues : programmes installés, exclusions de Defender, RecentDocs, MountPoints2, réseaux connus, configuration des journaux, entre autres. Celles qui ne tracent aucune activité sont écartées, avec leur raison. Les fichiers que KapeTriage était seul à prendre sont collectés et convertis : navigateurs Chromium, WebCache et INetCache, sessions et onglets ouverts ou fermés, favicons, prédicteur de la barre d'adresse, chronologie Windows, raccourcis hors de Recent, la ruche du profil Default, `$Secure:$SDS`, `$Boot`, les traces WPP de Defender et les clés maîtresses DPAPI. Seuls les `desktop.ini` sont écartés, faute de valeur d'enquête.

Vitesse sur les petits artefacts
Pris un par un, Prefetch, Amcache et raccourcis se décodent plus vite avec l'outil EZ : voir le chapitre 3.

Interprétations prêtes à lire
Les outils EZ découpent et nomment certains champs : exécutable, date et description d'une application SRUM, noms des dossiers connus, extension d'un fichier. WAC donne la valeur brute, telle que Windows la stocke.

Limite

Ce que KAPE prend sans que WAC le prenne, ce sont surtout des fichiers d'applications dans les profils (chapitre 4). Ils ne correspondent à aucun artefact que WAC décode : leur intérêt dépend de l'enquête.

CHAPITRE 7

Formats de sortie

7.1 Ce que produit chaque outil

Outil	Formats	Remarque
AmcacheParser	CSV	8 fichiers, un par table
AppCompatCacheParser	CSV	
RBCmd	CSV	
SrumECmd	CSV	7 fichiers
EvtxECmd	CSV, JSON, XML	
JLECmd	CSV, JSON, HTML	
LECmd	CSV, JSON, HTML, XML	
MFTECmd	CSV, JSON, bodyfile	
PECmd	CSV, JSON, HTML	
RECmd	CSV, JSON	plus un CSV par greffon
SBECmd	CSV, JSON	
KAPE	copie des fichiers, journal de copie CSV	
WAC	JSON pour chaque artefact	un fichier par artefact, plus le manifeste scellé

7.2 Ce qui les distingue

Quatre outils sur onze ne produisent que du CSV. Chez les autres, le JSON est la transcription de la ligne CSV : une ligne de texte par enregistrement (JSON Lines), à plat.

Types
Chez les outils EZ, les nombres sont des chaînes : un compteur d'exécutions s'écrit `"RunCount":"458"`, un numéro d'enregistrement `"EventRecordId":"21572"`. WAC écrit des nombres, des booléens, des chaînes.

Structures
Chez les outils EZ, une liste devient une chaîne jointe par des virgules (les fichiers chargés d'un Prefetch, ses répertoires), et les données d'un événement sont un JSON échappé *dans* une chaîne (`Payload`), à décoder une seconde fois. WAC imbrique : les volumes d'un Prefetch

et leurs répertoires, les éléments d'un raccourci, les paires `Name`/`Value` d'un événement.

Dates
Chez les outils EZ, l'UTC n'est pas signalé (`2026-09-30 23:04:46`) et la précision dépend d'une option. WAC donne l'ISO 8601 complet, en UTC (…`Z`) et en heure locale avec son décalage (…`+02:00`).

Absence de valeur
Les outils EZ écrivent une valeur par défaut, ou une cellule vide. WAC n'émet pas le champ ; un artefact non collecté est déclaré tel (`NotRequested`, `NotCollected`), pour qu'une liste vide ne passe pas pour une absence de trace.

Provenance
Le manifeste de WAC relie chaque valeur à la pièce dont elle vient, avec ses empreintes et ses dates d'origine.

Console
Les outils EZ détaillent leur travail sur la console ; certains y écrivent le décodage lui-même quand aucun fichier n'est demandé. WAC n'y écrit que l'avancement : tout ce qu'il trouve est dans les fichiers.

> **À retenir**
> Pour un analyste, la différence se voit au premier traitement : un JSON de WAC se charge tel quel dans un outil d'analyse ou dans WAE, sans rien redécouper ; une sortie EZ se traite comme un tableur, dont il faut reconvertir les nombres, redécouper les listes et redécoder les données d'événement.

CHAPITRE 8

Ce que la confrontation a appris à WAC

Chaque défaut ci-dessous produisait un JSON valide et faux, ou incomplet. Tous sont corrigés, avec un contrôle qui les aurait pris.

Artefact	Défaut corrigé
$MFT	enregistrements d'extension périmés fusionnés avec la base
EVTX	flottants tronqués à 6 chiffres ; binaire en « 69 06 00 00 » ; entiers hexadécimaux et GUID au mauvais format ; ProcessingErrorData ignoré (21 événements sans données) ; événements des journaux vidés non lus
Raccourcis	URI collée au bloc qui la suit ; lecture décalée de 2 octets sans liste d'ID ; blocs ExtraData non lus (machine, suivi) ; taille de la cible non lue ; DROID nuls émis
Jump lists	nombre d'ouvertures et rang MRU non lus ; format de DestList version 1 non pris en charge ; nom d'application altéré
Shellbags	dossier perdu avec sa descendance ; date de la clé du dossier, rang et NodeSlot absents ; dossier « Accueil » sans nom ; arbre de `NTUSER.DAT` non lu (seul celui de `UsrClass.dat` l'était)
Amcache	2 tables sur 14 et 4 à 6 valeurs lues : ni SHA-1, ni taille, ni pilote, ni périphérique ; SHA-1 nommé `Sha1`, lu comme celui du fichier entier, devenu `AmcacheSha1`
ShimCache	tabulations des applications empaquetées remplacées par des espaces
SRUM	dates FILETIME laissées en nombres ; texte UTF-16 lu comme de l'ANSI
Identifiants connus	105 chemins de DLL de la table des GUID (253 occurrences) dont la génération avait pris `\n` ou `\r` pour un saut de ligne : `system32\ netcorehc.dll` devenu `system32 etcorehc.dll`
Registre	BAM limité aux comptes ayant un profil (un compte supprimé perdait ses exécutions) ; durée de premier plan de UserAssist non lue
Collecte	pièces dont le chemin dépasse 260 caractères une fois dans la consigne non écrites (482 sur le kit complet) : écrites désormais en chemin étendu

Artefact	Défaut corrigé
Lancement	`system("cls")` lançait `cmd.exe` sur la machine examinée (mesure des traces)
Harnais	un contrôle qui plantait interrompait tous les suivants sans échec visible ; les outils EZ lancés sur le magasin scellé lui-même — SrumECmd y a écrit, WAC a refusé la conversion suivante ; ils lisent désormais une copie

CHAPITRE 9

Limites de la comparaison

— Une seule machine, virtuelle, sous Windows 11 : certains artefacts y sont peu nombreux (2 clés USB, une recherche dans l'Explorateur). Les formats sont les mêmes sur une machine réelle, mais les cas rares n'y sont pas tous représentés.

— Les deux collectes se suivent au lieu d'être simultanées : les fichiers écrits entre elles ($MFT, journaux, Prefetch) diffèrent légitimement.

— Les outils EZ ont été lancés avec leurs options de précision maximale ; par défaut, ils perdent davantage.

— Les temps viennent d'une machine virtuelle et de son disque : ils comparent les outils entre eux, ils ne prédisent pas la durée sur une machine réelle, où le nombre d'exécutables et la vitesse du disque pèsent le plus.

— La mesure des traces se fait sur un clone sans réseau, en une exécution par outil : une écriture de Windows que le témoin n'a pas montrée n'est pas imputée à l'outil sans que Sysmon l'attribue à l'un de ses processus ; elle reste listée comme non attribuée dans les résultats de `footprint-diff.py`.

— L'audit de création de processus (Security 4688) n'est pas actif sur la machine de test : sur une machine où il l'est, chaque outil y laisse un événement par processus — un pour WAC, quinze pour KAPE et ses modules.

— La comparaison porte sur les artefacts communs. Ce que seul l'un des outils lit n'a pas de référence ici : son exactitude repose sur les contrôles du dossier de validation.